

Post-Quantum Secure API Communication: Architectures and Migration Strategies for Quantum-Resilient Systems

Subhajit Roy[✉]

Independent Researcher

Email: subhajitroy857@gmail.com

ORCID: [0009-0000-4896-0922](https://orcid.org/0009-0000-4896-0922)

Abstract—The advent of cryptographically relevant quantum computers threatens the asymmetric primitives – RSA, Diffie-Hellman, and elliptic-curve cryptography – that underpin the confidentiality and integrity of contemporary Application Programming Interface (API) communication. While large-scale quantum adversaries do not yet exist, the “harvest now, decrypt later” threat model means that data encrypted today under classical schemes may already be exposed to future cryptanalysis. This paper examines the practical migration of API security architectures toward post-quantum cryptography (PQC), focusing on the National Institute of Standards and Technology (NIST) standardized algorithms ML-KEM (FIPS 203), ML-DSA (FIPS 204), and SLH-DSA (FIPS 205). We propose a layered reference architecture for quantum-resilient API systems, evaluate hybrid key-exchange strategies for Transport Layer Security (TLS) 1.3, and analyze the downstream effects of PQC adoption on certificate chains, JSON Web Tokens (JWT), OAuth 2.0 token issuance, and API gateway design. We further present a phased migration framework – spanning algorithm inventory, hybrid deployment, and cryptographic agility – intended to guide practitioners through the transition without requiring a disruptive, single-step cutover. The paper concludes with an assessment of open engineering challenges, including increased key and signature sizes, handshake latency, and the operational complexity of maintaining dual classical/post-quantum trust chains during the transition period.

Index Terms—post-quantum cryptography, API security, hybrid TLS, ML-KEM, ML-DSA, cryptographic agility, NIST PQC, TLS 1.3, quantum-safe communication, applied cryptography

I. INTRODUCTION

Modern distributed systems rely overwhelmingly on API-mediated communication, secured at the transport layer by TLS and at the application layer by token-based authentication and authorization schemes such as JWT and OAuth 2.0. The cryptographic foundations of these mechanisms – RSA, Diffie-Hellman key exchange, and elliptic-curve variants thereof – derive their security from the computational hardness of integer factorization and the discrete logarithm problem. Shor’s algorithm demonstrates that a sufficiently large fault-tolerant quantum computer solves both problems in polynomial time [1], which would render these primitives insecure.

Although current estimates of when a cryptographically relevant quantum computer (CRQC) will exist vary considerably across the research community [2], the asymmetry between attacker and defender timelines creates an immediate practical

concern. An adversary capable of recording encrypted API traffic today need not possess a quantum computer at the time of capture; decryption can be deferred until quantum capability matures. This “harvest now, decrypt later” (HNDL) model means that any long-lived sensitive data transmitted over classically-secured channels today is already at risk, independent of when quantum hardware materializes.

In response, NIST finalized its first set of post-quantum cryptographic standards in 2024: ML-KEM for key encapsulation (FIPS 203, based on CRYSTALS-Kyber) [3], [4], ML-DSA for digital signatures (FIPS 204, based on CRYSTALS-Dilithium) [5], [6], and SLH-DSA for hash-based digital signatures (FIPS 205, based on SPHINCS+) [7], [8]. These standards provide the cryptographic primitives required for quantum-resistant systems, but their integration into existing API ecosystems – TLS termination, certificate authorities, token issuance services, and API gateways – requires deliberate architectural planning.

This paper addresses the system-level question that follows standardization: *how should existing API infrastructure be re-architected and migrated to incorporate post-quantum primitives without disrupting availability, interoperability, or backward compatibility?* We do not propose new cryptographic constructions; rather, we synthesize current standards guidance, hybrid key-exchange practice, and migration literature into a practical reference architecture and phased migration framework intended for engineering teams operating production API systems.

A. Research Questions

This work is organized around four research questions:

- 1) **RQ1:** What architectural layers of an API system are affected by the transition to post-quantum cryptography, and what are the trust boundaries between them?
- 2) **RQ2:** How should hybrid key exchange be deployed in TLS 1.3 to provide quantum resistance while preserving compatibility with classical clients during the transition period?
- 3) **RQ3:** What changes are required in certificate chains, JWT issuance, and OAuth 2.0 flows to accommodate post-quantum signature schemes, and what are the practical costs of these changes?

- 4) **RQ4:** What migration sequencing minimizes operational risk while ensuring cryptographic agility for future algorithm transitions?

B. Key Contributions

- A five-layer reference architecture for quantum-resilient API systems, delineating trust boundaries between transport, certificate, token, gateway, and application layers.
- A threat model for API communication under both classical and HNDL quantum adversaries, identifying protected assets and attack surfaces specific to API ecosystems.
- An analysis of hybrid TLS 1.3 key-exchange deployment using ML-KEM alongside classical elliptic-curve Diffie-Hellman (ECDHE), following IETF hybrid key-exchange guidance [9].
- A phased migration framework covering classical-to-hybrid TLS transition, certificate chain migration, JWT/OAuth 2.0 signature migration, and API gateway re-architecture.
- A discussion of cryptographic agility as a design principle, enabling future algorithm transitions without structural re-architecture.

II. BACKGROUND

A. The Quantum Threat to Asymmetric Cryptography

RSA and Diffie-Hellman-based key exchange, including its elliptic-curve variants (ECDH, ECDHE), rely on problems – integer factorization and the discrete logarithm problem – that admit no known efficient classical solution. Shor’s algorithm [1] solves both in polynomial time on a quantum computer, which would allow an adversary to recover private keys from public keys and break both confidentiality (key exchange) and authenticity (digital signatures) for any system relying on these primitives. Symmetric primitives such as AES are comparatively less affected; Grover’s algorithm provides only a quadratic speedup, which is typically mitigated by doubling key length.

B. NIST Post-Quantum Standardization

NIST’s PQC standardization project [10] selected lattice-based and hash-based candidates as the first standardized algorithms:

- **ML-KEM (FIPS 203)** [3], derived from CRYSTALS-Kyber [4], provides key encapsulation based on the Module Learning With Errors (MLWE) problem.
- **ML-DSA (FIPS 204)** [5], derived from CRYSTALS-Dilithium [6], provides digital signatures based on a combination of MLWE and the Module Short Integer Solution (MSIS) problem.
- **SLH-DSA (FIPS 205)** [7], derived from SPHINCS+ [8], provides a stateless hash-based signature scheme with security resting solely on the collision resistance of the underlying hash function, serving as a conservative fallback should lattice assumptions weaken.
- **FALCON** [11], a NTRU-lattice-based signature scheme offering compact signatures, remains under continued

standardization consideration as a complementary signature algorithm.

Table I summarizes representative key and signature sizes relative to classical equivalents at comparable security levels.

TABLE I
APPROXIMATE KEY AND SIGNATURE SIZES (SECURITY CATEGORY 1, ≈ 128 -BIT)

Scheme	Public Key	Private Key	Ciphertext/Sig
ECDH (X25519)	32 B	32 B	32 B
ML-KEM-512	800 B	1632 B	768 B
ECDHSA (P-256)	64 B	32 B	64–72 B
ML-DSA-44	1312 B	2560 B	2420 B
SLH-DSA-128s	32 B	64 B	7856 B

The order-of-magnitude increase in key and signature sizes, particularly for SLH-DSA, has direct implications for TLS handshake size, certificate chain length, and JWT payload size – each discussed in Section IV.

C. Hybrid Cryptography as a Transition Strategy

Rather than replacing classical primitives outright, current IETF and NSA guidance favors *hybrid* key exchange during the transition period [9], [12]. A hybrid key exchange combines a classical algorithm (e.g., X25519) with a post-quantum algorithm (e.g., ML-KEM) such that the resulting shared secret is secure if *either* component remains unbroken. This approach hedges against two risks simultaneously: undiscovered weaknesses in newly standardized post-quantum schemes, and the eventual arrival of a CRQC capable of breaking classical schemes. We adopt hybrid key exchange as the primary transport-layer mechanism in the architecture proposed in Section IV.

III. THREAT MODEL

A complete threat model for this work is maintained in docs/threat-model.md within the accompanying repository; this section summarizes its core elements.

A. Adversary Capabilities

We consider two adversary classes:

- **Classical network adversary:** capable of passive eavesdropping, active man-in-the-middle positioning, and certificate or token forgery attempts bounded by classical computational limits.
- **Harvest-now, decrypt-later (HNDL) adversary:** passively records encrypted traffic and certificate material today, with the intent of applying quantum cryptanalysis once a CRQC becomes available. This adversary is bounded only by data retention capacity, not by present-day computational limits.

B. Protected Assets

The assets in scope are: (i) the confidentiality of data in transit between API clients and servers, (ii) the integrity and authenticity of API requests and responses, (iii) the long-term validity of certificate chains and trust anchors, and (iv) the

integrity of bearer tokens (JWT) used for authentication and authorization.

C. Security Goals

The architecture in Section IV is designed to achieve: forward secrecy against both classical and HNDL adversaries at the transport layer; unforgeability of certificates and tokens against quantum-capable signature forgery; and backward compatibility with classical-only clients during the migration period, without weakening the security posture for quantum-resistant clients.

IV. REFERENCE ARCHITECTURE

We propose a five-layer architecture for reasoning about post-quantum migration in API systems, illustrated in the API architecture diagram accompanying this repository ([diagrams/api-architecture.svg](#)).

- 1) **Transport Layer:** TLS 1.3 connection establishment, including the key-exchange mechanism (classical, post-quantum, or hybrid).
- 2) **Certificate Layer:** X.509 certificate issuance and validation, including the signature algorithm used by certificate authorities and the structure of certificate chains during dual-algorithm operation.
- 3) **Token Layer:** Application-level authentication and authorization artifacts – JWTs and OAuth 2.0 access/refresh tokens – and the signature or MAC scheme used to protect them.
- 4) **Gateway Layer:** API gateway and reverse proxy components responsible for TLS termination, request routing, and token validation, which must support algorithm negotiation and fallback.
- 5) **Application Layer:** Business logic consuming validated requests, generally agnostic to the underlying cryptographic scheme provided the lower layers expose a stable interface.

A diagrammatic representation of this architecture, the corresponding hybrid TLS handshake, and the cryptographic dependency graph are provided in [diagrams/](#) and described in [docs/architecture.md](#).

A. Trust Boundaries

Each layer boundary represents a point where cryptographic material crosses from one trust domain to another: the transport layer boundary is the TLS session itself; the certificate layer boundary is the handoff between certificate authority and relying party; the token layer boundary is the handoff between identity provider and resource server; and the gateway boundary mediates between external, potentially hybrid-capable clients and internal services that may still operate classical-only cryptography during migration.

B. Cryptographic Agility

We treat cryptographic agility – the capacity to substitute algorithms without redesigning system architecture – as a first-class design requirement rather than an afterthought. Practically, this is realized through: algorithm identifiers negotiated

at the protocol level (TLS cipher suites, JWT alg headers), configuration-driven algorithm selection in gateway and token-issuance components, and avoidance of hard-coded key or signature size assumptions in downstream application code.

V. HYBRID TLS FOR API TRANSPORT SECURITY

Hybrid key exchange in TLS 1.3 combines an ML-KEM encapsulation with a classical ECDHE exchange within the `key_share` extension, following the IETF hybrid design draft [9]. The combined shared secret is derived via a key-derivation function (KDF) that mixes both component secrets, ensuring that the session remains secure if either the classical or the post-quantum component is broken, but not necessarily if both are independently broken in different ways – a property generally referred to as IND-CCA2 security of the combiner.

A. Handshake Size and Latency

The principal engineering cost of hybrid key exchange is increased handshake size: ML-KEM-768 key shares are on the order of 1.1–1.2 KB versus 32 bytes for X25519, increasing the TLS `ClientHello` and `ServerHello` payloads correspondingly. For most API workloads using persistent or pooled connections, this one-time handshake cost is amortized over the connection lifetime and has negligible effect on steady-state throughput; latency-sensitive, connection-per-request architectures should weight this cost more heavily in capacity planning.

B. Server and Client Negotiation

Servers supporting hybrid TLS advertise hybrid key-share groups alongside classical groups, allowing clients without post-quantum support to negotiate a purely classical handshake. This preserves interoperability during the transition but means the security guarantee is only as strong as the weaker party’s capability; gateway-layer policy (Section IV) should be used to enforce minimum acceptable key-exchange groups for sensitive endpoints once client-side support is sufficiently mature.

VI. CERTIFICATE, JWT, AND OAUTH 2.0 MIGRATION

A. Certificate Chain Migration

Migrating X.509 certificate chains to post-quantum signature algorithms (ML-DSA, SLH-DSA, or FALCON) can proceed via two non-exclusive approaches: sequential migration, in which a certificate authority issues a new post-quantum-signed chain in parallel with the existing classical chain, and composite certificates, in which a single certificate embeds both a classical and a post-quantum public key and signature [13]. Composite certificates avoid the operational overhead of maintaining two parallel chains but require relying-party software capable of parsing the composite structure.

B. JWT and OAuth 2.0 Signature Migration

JWTs [14] and the OAuth 2.0 framework [15] are signature-algorithm-agnostic by design, identifying the signing algorithm in the JWT `alg` header. This permits incremental migration: an identity provider may begin issuing ML-DSA-signed tokens while resource servers retain the capacity to validate both classical (RS256/ES256) and post-quantum signatures during a defined transition window. The principal practical concern is token size: ML-DSA signatures (roughly 2.4 KB for ML-DSA-44) are substantially larger than ECDSA signatures (64–72 bytes), which has implications for HTTP header size limits when tokens are transmitted as bearer tokens in the Authorization header.

C. API Gateway Migration

API gateways sit at the convergence point of transport, certificate, and token layer migrations, and therefore require coordinated configuration changes: enabling hybrid TLS key-exchange groups, trusting dual-algorithm certificate chains, and validating tokens signed under either classical or post-quantum algorithms. We recommend gateway configuration be externalized (rather than hard-coded) to support algorithm rollback during the transition, consistent with the cryptographic agility principle in Section IV.

VII. MIGRATION FRAMEWORK

We propose a four-phase migration framework, detailed further in `docs/migration-guide.md`:

- 1) **Phase 0 – Inventory:** Catalog all cryptographic dependencies across transport, certificate, token, and gateway layers, including third-party libraries and vendor-managed infrastructure not under direct organizational control.
- 2) **Phase 1 – Hybrid Transport:** Enable hybrid TLS key exchange at gateway and load-balancer tiers, defaulting to classical-only fallback for non-upgraded clients.
- 3) **Phase 2 – Certificate and Token Migration:** Issue composite or parallel post-quantum certificate chains; begin dual-algorithm JWT issuance and validation.
- 4) **Phase 3 – Agility Consolidation:** Remove deprecated classical-only code paths once client population and compliance requirements permit, retaining configuration-level agility for future algorithm transitions.

This sequencing prioritizes transport-layer protection first, since HNDL exposure (Section IV) accrues continuously from the present moment, while certificate and token migration – which affect authenticity rather than confidentiality – can follow on a slightly longer timeline without retroactive exposure.

VIII. DISCUSSION AND OPEN CHALLENGES

Several engineering challenges remain open. First, the increased size of post-quantum keys, ciphertexts, and signatures (Table I) compounds across layered protocols – a TLS handshake, certificate chain, and bearer token may each independently grow, with cumulative effects on middlebox

compatibility (e.g., devices that assume bounded TLS record sizes) that are not yet fully characterized in production environments. Second, composite certificate formats remain in active IETF standardization [13] and are not yet uniformly supported across TLS libraries, creating near-term interoperability risk. Third, maintaining dual classical/post-quantum trust chains during the transition period introduces operational complexity – key management, certificate revocation, and audit processes must account for two parallel algorithm families simultaneously. Finally, cryptographic agility, while a sound design principle, is not free: configuration-driven algorithm selection introduces additional testing surface and the risk of misconfiguration (e.g., inadvertently permitting fallback to weak classical algorithms in production).

This paper does not provide original empirical performance measurements; the prototype, benchmark, and experimental evaluation components described in the accompanying repository are scoped as future work, intended to quantify the latency, throughput, and packet-size effects discussed qualitatively above.

IX. CONCLUSION

The transition to post-quantum cryptography in API systems is not a single algorithm substitution but a multi-layer architectural migration spanning transport, certificate, token, and gateway components. We have presented a five-layer reference architecture, a threat model accounting for both classical and harvest-now-decrypt-later adversaries, and a phased migration framework that prioritizes hybrid transport-layer protection while allowing certificate and token migration to proceed on a coordinated but distinct timeline. Cryptographic agility – the capacity to substitute algorithms without re-architecting the system – should be treated as a design requirement throughout this process, given the likelihood of continued evolution in post-quantum standards. Future work, outlined in the accompanying repository’s research roadmap, includes empirical benchmarking of hybrid TLS handshake latency, ML-KEM and ML-DSA performance under representative API workloads, and prototype implementation of the gateway-layer migration patterns described in Section IV.

ACKNOWLEDGMENT

The author thanks the maintainers of the Open Quantum Safe project [16] for their continued work providing accessible reference implementations of NIST post-quantum algorithms, which inform the prototype work planned in this repository’s research roadmap.

REFERENCES

- [1] P. W. Shor, “Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer,” *SIAM Journal on Computing*, pp. 1484–1509, 1997.
- [2] M. Mosca, “Cybersecurity in an Era with Quantum Computers: Will We Be Ready?” *IEEE Security & Privacy*, pp. 38–41, 2018.
- [3] National Institute of Standards and Technology, “FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard,” NIST, 2024.

- [4] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM,” in *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2018, pp. 353–367.
- [5] National Institute of Standards and Technology, “FIPS 204: Module-Lattice-Based Digital Signature Standard,” NIST, 2024.
- [6] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme,” in *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, vol. 2018, no. 1, 2018, pp. 238–268.
- [7] National Institute of Standards and Technology, “FIPS 205: Stateless Hash-Based Digital Signature Standard,” NIST, 2024.
- [8] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe, “SPHINCS+: Stateless Hash-Based Signatures,” NIST Post-Quantum Cryptography Standardization Submission, 2019.
- [9] IETF TLS Working Group, “Hybrid Key Exchange in TLS 1.3,” IETF Internet-Draft, draft-ietf-tls-hybrid-design, 2024.
- [10] National Institute of Standards and Technology, “Post-Quantum Cryptography Standardization Project,” <https://csrc.nist.gov/projects/post-quantum-cryptography>, 2024.
- [11] P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang, “FALCON: Fast-Fourier Lattice-Based Compact Signatures over NTRU,” in *NIST Post-Quantum Cryptography Standardization Round 3 Submission*, 2020.
- [12] National Security Agency, “Commercial National Security Algorithm Suite 2.0 (CNSA 2.0),” U.S. National Security Agency Advisory, 2022.
- [13] IETF LAMPS Working Group, “Composite Public and Private Keys for X.509 Certificates,” IETF Internet-Draft, draft-ietf-lamps-pq-composite-keys, 2024.
- [14] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, 2015.
- [15] D. Hardt, “The OAuth 2.0 Authorization Framework,” IETF RFC 6749, 2012.
- [16] Open Quantum Safe Project, “liboqs: C Library for Quantum-Resistant Cryptographic Algorithms,” <https://openquantumsafe.org>, 2024.